

SOFTWARE ENGINEERING

05

Database Management – SQL & PL/SQL

Store it. Query it. Trust it — every great app runs on a solid database backbone.

DURATION	LEVEL	MODULE	UPDATED
9 Hours · 9 Sessions + 1 Mini Project	Intermediate	Module 5	May 2026

IN THIS MODULE

01	Introduction to Databases
02	SQL Basics & Commands
03	DML Operations
04	Data Retrieval with SELECT
05	Joins and Relationships
06	Grouping and Subqueries
07	Constraints & Views
08	Introduction to PL/SQL
09	Transactions & Error Handling
10	Mini Project 3 – Expense Tracker DB

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to Databases

◆ Learning Objectives

- Define database, DBMS, and RDBMS using real-world analogies
- Distinguish between flat-file storage (Excel) and a relational database
- Compare MySQL, PostgreSQL, Oracle, and SQLite by use case and who uses them
- Explain what tables, rows, columns, primary keys, and foreign keys represent
- Navigate MySQL Workbench and locate a sample database table
- Connect database concepts to apps used daily — Swiggy, IRCTC, Zerodha

01 What Is Data — and Why Store It Properly?

Every time you order food on Swiggy, three things happen in under a second: your name is saved, your address is fetched, and your order history is updated. Behind all of that is a database — an organised collection of related data that can be stored, searched, and updated reliably. Without a database, Swiggy would need to ask for your address every single time.

CONCEPT — Database

A database is a structured collection of related data stored so it can be easily accessed, managed, and updated. Think of it like a digital filing cabinet — one that can answer 'Show all orders above Rs 500 placed in Ahmedabad this week' in under a millisecond, even with a crore records inside.

Flat File (Excel / CSV)

Data lives in rows and columns in one file. No relationships between files. Duplicates pile up quickly. Searching large data is slow. Two users editing at the same time causes overwrites and conflicts.

Relational Database

Data is split into related tables. Relationships are enforced by keys. Duplicates are eliminated by design. Queries run in milliseconds on crores of rows. Built to handle thousands of simultaneous users without data loss.

02 DBMS and RDBMS

CONCEPT — DBMS vs RDBMS

A DBMS (Database Management System) is software that manages stored data — early examples were simple file-based systems. An RDBMS (Relational DBMS) is a DBMS that organises data into tables with defined relationships between them. MySQL, PostgreSQL, and Oracle are all RDBMS tools. Every RDBMS is a DBMS — but not every DBMS is an RDBMS.

In an RDBMS, data follows a strict structure called a schema — a blueprint that tells the database exactly what kind of data each column accepts: numbers, text, or dates. This prevents bad or incomplete data from entering the system. IRCTC, which handles over 1.4 million ticket bookings per day, runs on Oracle RDBMS precisely to guarantee data accuracy across millions of simultaneous users.

03 Popular Database Systems

Database	Type	Best For	Who Uses It
MySQL	RDBMS (open source)	Web apps, startups, learning	WordPress, early Facebook, TOPS projects
PostgreSQL	RDBMS (open source)	Complex queries, strict data integrity	Zerodha, Swiggy backend layers
Oracle DB	RDBMS (commercial)	Enterprise, banking, government	IRCTC, SBI, large Indian banks
SQLite	RDBMS (file-based)	Mobile apps, local offline storage	Android apps, WhatsApp local cache

PRO TIP — Which One to Learn First?

Start with MySQL — it is free, beginner-friendly, and listed in the majority of fresher developer job descriptions on Naukri.com. Once you know MySQL, switching to PostgreSQL or Oracle takes only a few days. This entire module uses MySQL.

04 Tables, Rows, Columns & Keys

A relational database stores data in tables — structurally similar to a spreadsheet, but far more powerful. Each table has columns (what type of data) and rows (actual records). A users table might have columns: user_id, name, email, city — and each row is one real person. A separate expenses table holds every transaction, linked back to the correct user by user_id.

CONCEPT — Primary Key & Foreign Key

A Primary Key uniquely identifies every row in a table — no two rows can share the same value. Example: user_id = 1001 belongs to exactly one person. A Foreign Key is a column in one table that references the Primary Key in another, creating a relationship. In an expenses table, user_id = 1001 always points back to that specific person — so you always know who spent what.

TRY THIS LIVE

Open MySQL Workbench -> connect to your local MySQL server -> expand the 'sakila' sample database in the left panel -> expand Tables -> right-click any table -> Select Rows – Limit 1000. You are now viewing real data inside a real database table.

05 Trainer Demo: First Look at MySQL Workbench

Before writing any queries, get comfortable navigating MySQL Workbench. The following commands are the first five you should know — they let you orient yourself inside any MySQL server, see what exists, and inspect the structure of any table.

<code>SHOW DATABASES;</code>	List all databases on this server
<code>USE sakila;</code>	Switch into the 'sakila' sample database
<code>SHOW TABLES;</code>	List all tables inside the current database
<code>DESCRIBE actor;</code>	Show the column structure of the 'actor' table
<code>SELECT * FROM actor LIMIT 5;</code>	Preview the first 5 rows — quick data snapshot

CASE STUDY — IRCTC & Oracle RDBMS

Context — IRCTC (Indian Railway Catering and Tourism Corporation) is India's official railway ticketing platform, serving 23 million passengers per day across 13,000+ trains. It is one of the most visited websites in India.

Challenge — During Diwali and Holi seasons, millions of users simultaneously attempt to book the same limited seats. A flat-file system would crash immediately under this load — and any data corruption means a passenger could lose a confirmed seat with no record of payment.

What they did — IRCTC runs on Oracle RDBMS with a schema that separates trains, stations, passengers, bookings, and payments into dedicated related tables. Primary and foreign keys link every booking to a specific passenger and a specific train, making orphaned or duplicate records structurally impossible.

Result — The system handles 1.4 million+ bookings daily with no data corruption — even during load spikes that occasionally cause frontend outages (those are server capacity issues, not database failures).

Takeaway — A well-structured relational schema with clearly defined primary and foreign keys is what allows a system to grow from 100 users to 100 million without losing a single record.

Source: IRCTC Annual Report 2023–24; Oracle India Customer Stories (oracle.com/customers)

DATABASES IN INDIAN TECH

8 in 10

Top Indian unicorn startups use MySQL or PostgreSQL as their primary database. Source: BuiltWith India Tech Stack

Analysis, 2024

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Database Management System — Lecture 1: Introduction to DBMS. Delivered by IIT Kharagpur faculty. Navigate to Week 1 on the course page and confirm the lecture number before sharing.

Link — <https://nptel.ac.in/courses/106105175>

Always link the official channel — beware fan-made or re-upload copies.

“Data is the raw material. A database is the factory that turns it into something useful.”

WHERE TO GO NEXT

You now understand what a database is and why structure matters — next, you will learn to speak its language.

Explore deeper — Read: 'What is a Relational Database?' — cloud.google.com/learn/what-is-a-relational-database

Try on your own — Open MySQL Workbench, browse every table inside the 'sakila' sample database, and sketch a rough diagram connecting related tables — pen and paper is perfectly fine.

Connect the dots — Session 2 picks up directly here — you will write your first DDL commands (CREATE, ALTER, DROP) to build the users and expenses tables from scratch.

Going further — Search 'database normalisation 1NF 2NF 3NF' — the design rules that keep relational databases fast and free of duplicated data at scale.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a database teacher. Explain the difference between DBMS and RDBMS in simple Hinglish, using the example of a school managing student records. Tell me why Excel fails when data grows to 500+ students. Keep it under 150 words.

SESSION

02

60 MIN · SQL FUNDAMENTALS

SQL Basics & Commands

◆ Learning Objectives

- Define SQL and explain DDL, DML, and DQL as the three main command categories
- Identify the correct MySQL data type for different kinds of information
- Write and execute a CREATE TABLE statement with appropriate data types and constraints
- Modify an existing table structure using ALTER TABLE without losing existing data
- Remove a table using DROP TABLE and explain the permanence of this action
- Build the users and expenses tables that will be used throughout the rest of Module 5

01 SQL — The Language Databases Speak

SQL (Structured Query Language) is the standard language for communicating with a relational database. Whether you use MySQL, PostgreSQL, or Oracle, the core SQL commands are the same — similar to how standard Hindi works whether you are in Ahmedabad, Mumbai, or Delhi, even though each city has its own flavour. SQL was standardised in 1986 and remains the most requested technical skill in Indian data and backend developer job listings.

CONCEPT — DDL, DML, and DQL

SQL commands are grouped into three categories. DDL (Data Definition Language) defines the structure — CREATE, ALTER, DROP. DML (Data Manipulation Language) works with the data inside — INSERT, UPDATE, DELETE. DQL (Data Query Language) retrieves data — SELECT. This session focuses entirely on DDL: building the database skeleton before any data enters it.

Category	Full Name	Key Commands	What It Does
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Defines or changes the structure of tables
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Adds, modifies, or removes data inside tables
DQL	Data Query Language	SELECT	Retrieves and filters data from tables
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manages transactions — covered in Session 9

02 CREATE TABLE — Defining Your First Tables

CREATE TABLE defines the blueprint of a table — its name, columns, data types, and constraints. Before you can store a single user record or a single expense, the table must exist. Think of it like setting up a new register: you draw the column headings first, then start filling in the rows. Choose your data types carefully — changing them later is possible but disruptive.

Data Type	What It Stores	Use in Our Project
INT	Whole numbers	user_id, expense_id
DECIMAL(10, 2)	Numbers with decimal places	amount — e.g. 1500.75 (never use FLOAT for money)
VARCHAR(n)	Text up to n characters	name, email, category, city
DATE	Date only — YYYY-MM-DD	expense_date
DATETIME	Date and time together	created_at — auto-filled on insert
TINYINT(1)	True / False stored as 1 or 0	is_active, is_recurring

◆ Users Table

```
CREATE TABLE users (
  user_id    INT AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  email      VARCHAR(150) UNIQUE NOT NULL,
  city       VARCHAR(50),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  Auto-fills with current date and time on every INSERT
);
```

Start defining the users table

Unique row ID — auto-increments; no manual entry needed

Text, max 100 chars; cannot be left empty

Must be unique across all rows; cannot be empty

Optional — NULL is allowed if city is not provided

Close and execute the statement

◆ Expenses Table

```
CREATE TABLE expenses (
  expense_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id    INT NOT NULL,
  category   VARCHAR(50) NOT NULL,
  amount     DECIMAL(10, 2) NOT NULL,
```

Start defining the expenses table

Unique ID for each expense record

Links every expense to a user — required field

e.g. Food, Transport, Entertainment

Rupee value with 2 decimal places — never FLOAT for money

```
expense_date DATE NOT NULL,
notes VARCHAR(255),
FOREIGN KEY (user_id) REFERENCES users(user_id)
    Enforces the link — user_id must exist in users table
);
```

Date the expense occurred — format: YYYY-MM-DD
Optional short description of the expense
Close and execute

TRY THIS LIVE

Open MySQL Workbench -> open a new SQL tab -> paste and run the users table first, then the expenses table (the foreign key in expenses depends on users existing). Expand your schema in the left panel — both tables should now appear. Right-click each -> Table Inspector to confirm all column names, types, and constraints.

03 ALTER TABLE — Modifying Structure

Once a table is created, requirements will change — a new column is needed, a VARCHAR length is too short, or a column was named incorrectly. ALTER TABLE lets you modify a table's structure without deleting it or touching any data already inside. In real projects, ALTER TABLE is one of the most-used DDL commands, because no schema survives first contact with real users unchanged.

```
ALTER TABLE users ADD phone VARCHAR(15);
ALTER TABLE users MODIFY city VARCHAR(100);
ALTER TABLE users RENAME COLUMN city TO location;
ALTER TABLE users DROP COLUMN phone;
```

Add a new 'phone' column — allows NULL by default
Expand city column from 50 to 100 characters
Rename 'city' to 'location' — MySQL 8.0 and above only
Remove the phone column — any data in it is lost permanently

04 DROP TABLE — Removing with Caution

WARNING — DROP is Permanent

DROP TABLE removes the table AND every row of data inside it instantly — there is no Recycle Bin, no Undo. In production databases, DROP access is restricted to senior engineers and always requires a full backup first. In your learning environment, always double-check the table name before running DROP.

```
DROP TABLE expenses;
DROP TABLE IF EXISTS expenses;
```

Deletes the entire expenses table and all its data — permanent
Safer — only drops if the table exists; avoids an error otherwise

TRUNCATE TABLE expenses;

Removes all rows but keeps the table structure — faster than DELETE for clearing data

CASE STUDY — Zerodha & Strict Schema Design

Context — Zerodha is India's largest stockbroker by active clients (7+ million as of 2024), headquartered in Bengaluru. Their engineering team runs one of the highest-transaction-volume PostgreSQL deployments in India.

Challenge — Every trade touches multiple tables — orders, holdings, margin, ledger — simultaneously. Using FLOAT instead of DECIMAL for money causes rounding errors worth crores. A missing NOT NULL constraint allows empty order records to be created, silently corrupting portfolio calculations.

What they did — Zerodha's engineers documented their schema design decisions on zerodha.tech — using DECIMAL for all financial values, enforcing NOT NULL and UNIQUE constraints at the database level rather than only in application code, and restricting DROP access in production to senior engineers with a mandatory backup checkpoint before any structural change.

Result — Zerodha processes millions of trades daily with near-zero data discrepancies. Their schema design post became a widely referenced guide for Indian fintech startups building their first databases.

Takeaway — Every constraint written in CREATE TABLE is a permanent guardrail. It is far cheaper to enforce data rules in the schema than to clean up corrupt records months later.

Source: Zerodha Engineering Blog — zerodha.tech; SEBI DRHP filing, 2023

SQL IN THE INDIA JOB MARKET

#1

SQL is the most frequently requested technical skill in data and backend fresher job listings. Source: Naukri.com Hiring Outlook Report, 2024

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Database Management System — SQL DDL Lectures (IIT Kharagpur). Navigate to the SQL section (approximately Weeks 2–3) on the course page and confirm lecture numbers before sharing with students.

Link — <https://nptel.ac.in/courses/106105175>

“A schema written carefully today saves a data-cleanup weekend six months from now.”

WHERE TO GO NEXT

The skeleton is built — two clean, constrained tables ready to receive data. Now it is time to fill them.

Explore deeper — Read: MySQL 8.0 Data Types reference — dev.mysql.com/doc/refman/8.0/en/data-types.html

Try on your own — Use ALTER TABLE to add two more columns to your expenses table: payment_mode VARCHAR(30) and is_recurring TINYINT(1) DEFAULT 0.

Connect the dots — Session 3 introduces DML — INSERT, UPDATE, DELETE — to populate and modify the exact tables you built today.

Going further — Look up 'Flyway' and 'Liquibase' — the tools professional dev teams use to manage ALTER TABLE changes safely across development, staging, and production environments.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a MySQL instructor. Explain CREATE TABLE, ALTER TABLE, and DROP TABLE in simple Hinglish. Use the example of building an expense tracker app for college students in Gujarat. Tell me why DECIMAL is better than FLOAT for storing money. Keep it under 150 words.

SESSION

03

60 MIN · SQL FUNDAMENTALS

DML Operations

◆ Learning Objectives

- Insert a single row into a table using INSERT INTO with explicit column names
- Insert multiple rows in one statement for efficiency
- Update specific rows using UPDATE with a WHERE clause to target only the intended records
- Delete specific rows using DELETE with WHERE and LIMIT to prevent accidental data loss
- Explain why running UPDATE or DELETE without a WHERE clause is a critical risk
- Build a dataset of 5 users and 5 expenses that will be used in Sessions 4 through 9

01 INSERT INTO — Adding New Records

DDL created the empty tables — like printing a blank register. DML is what actually writes in it. INSERT INTO adds new rows to a table. In the Expense Tracker, every new student registration runs an INSERT INTO users, and every payment they log runs an INSERT INTO expenses. You can insert one row at a time or many rows in a single statement.

CONCEPT — INSERT INTO Syntax

INSERT INTO table_name (col1, col2) VALUES (val1, val2). The column list tells MySQL which fields you are filling; the VALUES list provides the data in the same order. Columns not listed will use their DEFAULT or NULL. Always name your columns explicitly — skipping them breaks the query the moment the table structure changes.

◆ Single Row — First User

```
INSERT INTO users (name, email, city)          Specify the table and columns to fill
VALUES ('Riya Shah', 'riya@example.com', 'Ahmedabad');
```

Data for one row — values match the column order above

◆ Multi-Row — Remaining Users

```
INSERT INTO users (name, email, city) VALUES      One statement, four rows — more efficient than four
                                                    separate INSERTs

('Arjun Mehta', 'arjun@example.com', 'Surat'),
    Row 2 — user from Surat
('Priya Patel', 'priya@example.com', 'Rajkot'),
    Row 3 — user from Rajkot
```

```
('Kabir Joshi', 'kabir@example.com', 'Ahmedabad'),
Row 4 — second Ahmedabad user
('Sneha Desai', 'sneha@example.com', 'Vadodara');
Row 5 — semicolon closes the entire statement
```

◆ Insert Expenses

```
INSERT INTO expenses (user_id, category, amount, expense_date, notes) VALUES
```

Insert 5 sample expense records

```
(1, 'Food', 450.00, '2025-06-01', 'Canteen lunch'),
Riya's food expense — user_id = 1 links to Riya
(1, 'Transport', 120.00, '2025-06-01', 'Auto rickshaw fare'),
Riya's transport on the same day
(2, 'Food', 680.00, '2025-06-02', 'Dinner with friends'),
Arjun's food expense — user_id = 2
(3, 'Books', 850.00, '2025-06-03', 'Reference book'),
Priya's book purchase — user_id = 3
(4, 'Transport', 200.00, '2025-06-01', 'Bus pass top-up');
Kabir's transport — user_id = 4
```

TRY THIS LIVE

Run the single-row INSERT first, then the multi-row INSERT for users, then the expenses INSERT. After each statement, run `SELECT * FROM users;` and `SELECT * FROM expenses;` to confirm the rows appeared. You should now see 5 users and 5 expenses — this dataset is used for every remaining session in Module 5.

02 UPDATE — Modifying Existing Records

UPDATE changes values in one or more columns for rows that match a condition. The syntax has three parts: which table, which columns to change (SET), and which rows to affect (WHERE). You can target a single row by its primary key — the safest approach — or update many rows at once by matching a broader condition.

WARNING — Always Use WHERE with UPDATE

UPDATE without a WHERE clause changes every row in the table instantly — there is no undo. Before running any UPDATE, first run a SELECT with the same WHERE condition to preview exactly which rows will be affected. This single habit prevents the most common accidental data-corruption mistake in professional databases.

```
-- Preview first — always do this before UPDATE
Run SELECT to see what will be affected before changing anything
SELECT * FROM users WHERE user_id = 3;           Confirm this is the row you intend to update
UPDATE users SET city = 'Gandhinagar' WHERE user_id = 3;
Change Priya's city — one row targeted by primary key (safest)
UPDATE expenses SET amount = 500.00 WHERE expense_id = 1;
Correct Riya's first expense amount
UPDATE expenses SET category = 'Dining' WHERE category = 'Food';
Rename all Food rows to Dining — affects multiple rows at once
```

03 DELETE — Removing Records

DELETE removes rows from a table based on a condition. Unlike DROP TABLE (which removes the entire table structure), DELETE only removes matching rows and leaves the table intact for new data. LIMIT caps how many rows are deleted in one operation — a useful safety net when you are not fully confident in your WHERE condition.

WARNING — Always Use WHERE with DELETE

DELETE without WHERE removes every row in the table — the same effect as TRUNCATE, with no Recycle Bin. In professional teams, a DELETE without WHERE in any production script is treated as a critical bug. Preview with SELECT first, then run the DELETE. Use LIMIT as a last line of defence.

```
-- Preview first
Run this SELECT before the DELETE to confirm which rows match

SELECT * FROM expenses WHERE expense_id = 5;   Verify the row before deleting
DELETE FROM expenses WHERE expense_id = 5;     Remove one specific expense by primary key — safest approach

DELETE FROM expenses WHERE amount < 150.00;    Remove all expenses below Rs 150 — targets multiple rows

DELETE FROM expenses WHERE category = 'Dining' LIMIT 1;
Delete only the first matching Dining row — LIMIT acts as a safety cap
```

04 Trainer Demo: Verify Every DML Change

After every INSERT, UPDATE, or DELETE, always verify the result with a SELECT. In professional development, you never assume a DML statement worked — you confirm it. The queries below are the verification checks for the dataset you have just built.

```
SELECT * FROM users;
SELECT COUNT(*) AS user_count FROM users;
SELECT * FROM expenses;
SELECT * FROM expenses WHERE user_id = 1;
SELECT * FROM users WHERE city = 'Gandhinagar';
```

Confirm Priya's UPDATE applied — should return 1 row

Confirm all 5 users were inserted with correct data
 Quick count — should return 5 if all inserts ran correctly
 Confirm all expense records and check amounts
 View only Riya's expenses — filter by foreign key

CASE STUDY — Flipkart Big Billion Days & DML at Scale

Context — Flipkart, India's largest e-commerce platform, runs its annual Big Billion Days sale every October — one of the highest-traffic commerce events in the world, with buyers across every Indian state including Gujarat.

Challenge — During the sale, millions of orders are placed per hour. Each order requires an INSERT into the orders table, an UPDATE to reduce stock count in the products table, and — for cancellations — a status flag update. All of this must happen in real-time without any inconsistency.

What they did — Flipkart's engineering team uses a distributed MySQL cluster where INSERTs are batched and queued, inventory UPDATES use atomic transactions, and cancellations use soft-delete: a boolean column `is_cancelled = 1` is set instead of physically DELETing the row, preserving order history for customer service.

Result — Flipkart processed over 1 crore orders in the first few hours of Big Billion Days 2023 with no duplicate orders and no incorrect inventory counts — data integrity maintained throughout the peak load.

Takeaway — INSERT, UPDATE, and DELETE are the exact operations behind India's biggest shopping event — just executed millions of times per hour, wrapped in transactions and queued for safety.

Source: Flipkart Engineering Blog (tech.flipkart.com); Inc42 Big Billion Days 2023 coverage

DML AT FLIPKART SCALE

1 Cr+

Order records (INSERT operations) processed in the opening hours of Flipkart Big Billion Days 2023. Source: Flipkart Media Release, October 2023

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Database Management System — SQL DML: INSERT, UPDATE, DELETE (IIT Kharagpur).
 Navigate to the DML section on the course page and confirm lecture numbers before sharing with students.

Link — <https://nptel.ac.in/courses/106105175>

*“Always **SELECT** before you **UPDATE** or **DELETE**. One preview query can save an hour of data recovery.”*

WHERE TO GO NEXT

Your tables now have real data — next, you will ask intelligent questions of it.

Explore deeper — Read: 'MySQL INSERT Statement' — dev.mysql.com/doc/refman/8.0/en/insert.html

Try on your own — Add two more expense rows for Sneha (`user_id = 5`), then **UPDATE** one of them to correct the amount. Verify with `SELECT * FROM expenses WHERE user_id = 5`.

Connect the dots — Session 4 introduces **SELECT** in full — filtering, sorting, aliases, and aggregate functions — all using this exact dataset.

Going further — Research 'soft delete vs hard delete in database design' — most production systems never truly **DELETE** rows, and now you can see exactly why.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

*Act as a MySQL instructor. Explain **INSERT INTO**, **UPDATE**, and **DELETE** in simple Hinglish, using an example of a student logging monthly expenses at a college canteen in Ahmedabad. Tell me why running **UPDATE** or **DELETE** without **WHERE** is dangerous. Keep it under 150 words.*

SESSION

04

60 MIN · SQL QUERYING

Data Retrieval with SELECT

◆ Learning Objectives

- Write SELECT statements to retrieve specific columns from a table
- Filter rows using WHERE with comparison, range, pattern, and list operators
- Sort query results using ORDER BY in ascending and descending order
- Rename output columns for readability using aliases with AS
- Apply COUNT, SUM, AVG, MIN, and MAX to summarise and analyse data
- Generate a category-wise total expense report as the session demo

01 SELECT — Asking Questions of Your Data

SELECT is the most-used SQL command — it retrieves data from one or more tables without changing anything. Every dashboard, report, and search result in every Indian app you use is powered by a SELECT query behind the scenes. Zomato showing your order history, Groww displaying your portfolio, PhonePe listing your transactions — all SELECT.

CONCEPT — SELECT Syntax

SELECT col1, col2 FROM table_name; retrieves specific columns. SELECT * FROM table_name; retrieves all columns — useful for exploring, but avoid in production code as it fetches unnecessary data. Always be specific about which columns you need; it makes queries faster and code easier to maintain.

<code>SELECT * FROM users;</code>	All columns, all rows — quick exploration only
<code>SELECT name, email FROM users;</code>	Only the name and email columns — more efficient
<code>SELECT name, city FROM users WHERE city = 'Ahmedabad';</code>	Users from Ahmedabad only — combines column selection and filtering
<code>SELECT * FROM expenses LIMIT 3;</code>	First 3 rows only — useful for previewing large tables

02 Filtering Rows with WHERE

WHERE tells MySQL which rows to include in the result. Without WHERE, SELECT returns every row. With WHERE, you get only what you need — like asking 'show me only expenses above Rs 500' or 'show me only users from Surat'. You can combine multiple conditions using AND (both must be true) and OR (either can be true).

Operator	Meaning	Example on Expenses Table
=	Exactly equal to	WHERE category = 'Food'

Operator	Meaning	Example on Expenses Table
!= or <>	Not equal to	WHERE category != 'Transport'
> / <	Greater / Less than	WHERE amount > 500
>= / <=	Greater / Less than or equal	WHERE amount <= 1000
BETWEEN	Inclusive range (both ends included)	WHERE amount BETWEEN 200 AND 800
LIKE	Pattern match — % is a wildcard	WHERE notes LIKE '%lunch%'
IN	Matches any value in a list	WHERE category IN ('Food', 'Books')
IS NULL	Column has no value entered	WHERE notes IS NULL

```
SELECT * FROM expenses WHERE amount > 500;      Show expenses above Rs 500
SELECT * FROM expenses WHERE amount BETWEEN 100 AND 500;
    Show expenses between Rs 100 and Rs 500 — both ends inclusive
SELECT * FROM expenses WHERE category IN ('Food', 'Transport');
    Show only Food and Transport rows
SELECT * FROM expenses WHERE notes LIKE '%lunch%';
    Find expenses whose notes contain the word 'lunch'
SELECT * FROM expenses WHERE amount > 300 AND category = 'Food';
    Both conditions must be true — combine with AND
SELECT * FROM expenses WHERE notes IS NULL;      Find expense rows where no notes were entered
```

TRY THIS LIVE

Run this query: `SELECT * FROM expenses WHERE amount > 400 ORDER BY amount DESC;` — it should return only the Food and Books expenses from your dataset. Then change 400 to 200 and run again. Notice how the result set grows. This is how every filter in every app works.

03 Sorting Results and Using Aliases

ORDER BY sorts the result set by one or more columns. ASC (ascending, A to Z, low to high) is the default. DESC (descending) flips the order — useful for 'show most expensive first' or 'latest transaction at the top'. AS renames a column in the output — it does not change the table, only what you see in the result.

```
SELECT name AS student_name, city AS location
FROM users ORDER BY city ASC;
SELECT category, amount AS rupees FROM expenses
ORDER BY amount DESC LIMIT 3;
```

Rename columns in the result — useful for clean reports
Sort alphabetically by city A to Z
Alias 'amount' as 'rupees' in the output
Show only the 3 most expensive expenses, highest first

04 Aggregate Functions — Summarising Data

CONCEPT — Aggregate Functions

Aggregate functions collapse many rows into one summary value. Instead of returning a list of individual expense amounts, `SUM(amount)` returns the total. Instead of listing every user, `COUNT(*)` returns how many there are. These functions are what turn a database full of raw transactions into a meaningful financial report.

Function	What It Returns	Example on Expenses Table
<code>COUNT(*)</code>	Total rows (includes NULLs)	<code>COUNT(*)</code> -> 5 (all 5 expense rows)
<code>COUNT(column)</code>	Rows where the column is NOT NULL	<code>COUNT(notes)</code> -> rows with notes entered
<code>SUM(column)</code>	Total of all values in a numeric column	<code>SUM(amount)</code> -> total rupees spent
<code>AVG(column)</code>	Average value across all matching rows	<code>AVG(amount)</code> -> average expense amount
<code>MIN(column)</code>	Smallest value in the column	<code>MIN(amount)</code> -> cheapest expense
<code>MAX(column)</code>	Largest value in the column	<code>MAX(amount)</code> -> most expensive expense

```
SELECT COUNT(*) AS total_expenses FROM expenses;
-- How many expense rows exist in total

SELECT COUNT(notes) AS expenses_with_notes FROM expenses;
-- How many rows have notes — skips NULLs

SELECT SUM(amount) AS total_spent FROM expenses WHERE user_id = 1;
-- Total amount spent by Riya (user_id = 1)

SELECT AVG(amount) AS avg_expense FROM expenses;
-- Average expense amount across all records

SELECT MIN(amount) AS cheapest, MAX(amount) AS priciest FROM expenses;
-- Cheapest and most expensive expense in one query
```

05 Trainer Demo: Expense Report by Category

The demo query below uses GROUP BY to group rows with the same category together, then SUM adds the amounts within each group. This produces the 'total expenses by category' report. GROUP BY is covered in depth in Session 6 — for now, focus on how SUM and ORDER BY combine to produce a meaningful financial summary from raw rows.

<code>SELECT category,</code>	Show the category name in the result
<code> COUNT(*) AS num_transactions,</code>	How many expenses exist in each category
<code> SUM(amount) AS total_spent,</code>	Total rupees spent per category
<code> AVG(amount) AS avg_per_transaction</code>	Average expense size within each category
<code>FROM expenses</code>	From the expenses table
<code>GROUP BY category</code>	Collapse all rows sharing a category into one summary row
<code>ORDER BY total_spent DESC;</code>	Show the highest-spending category at the top

PRO TIP — COUNT(*) vs COUNT(column)

COUNT(*) counts every row regardless of NULLs — use it when you want the total number of records. COUNT(column) only counts rows where that column has a value — use it when NULLs are meaningful and you want to exclude them. On the expenses table: COUNT(*) = 5, but COUNT(notes) might be 3 if two rows have no notes.

CASE STUDY — Groww & SELECT-Driven Portfolio Dashboards

Context — Groww is one of India's fastest-growing investment platforms, with 10+ million active investors as of 2024, headquartered in Bengaluru. Their users include many first-time investors from cities like Ahmedabad and Surat.

Challenge — Every time a user opens their portfolio, the app must compute — in milliseconds — total invested (SUM), current value (SUM), average return percentage (AVG), and best/worst performing stock (MAX/MIN) across potentially thousands of individual holdings. Running this slowly would cause users to abandon the app.

What they did — Groww built their portfolio dashboard on PostgreSQL using SELECT queries with aggregate functions filtered by user_id. Frequently-accessed aggregates like total portfolio value are pre-computed and cached, while on-demand filters ('show only equity funds above Rs 10,000') run live SELECT + WHERE + ORDER BY queries against the holdings table.

Result — Portfolio dashboards load in under 2 seconds for most users, computing live aggregates across millions of rows. The same SQL aggregate patterns power Groww's analytics team's daily reports on user investment behaviour.

Takeaway — The SELECT, WHERE, ORDER BY, COUNT, SUM, AVG, MIN, and MAX patterns you learned today are exactly what powers every investment portfolio app, bank dashboard, and expense report in India.

Source: Groww Engineering Blog (engineering.groww.in); SEBI DRHP filing, 2021

SQL DAILY USAGE — GLOBAL

77%

Developers who use SQL queries in their work every day. Source: Stack Overflow Developer Survey, 2023

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Database Management System — SQL SELECT, WHERE, Aggregate Functions (IIT Kharagpur).
Navigate to the query section on the course page and confirm lecture numbers before sharing with students.

Link — <https://nptel.ac.in/courses/106105175>

“A database full of raw rows becomes useful the moment you learn to ask it the right questions.”

WHERE TO GO NEXT

You can now retrieve, filter, sort, and summarise data — next you will group and slice it even further.

Explore deeper — Read: 'MySQL Aggregate Functions' —
dev.mysql.com/doc/refman/8.0/en/aggregate-functions.html

Try on your own — Write a SELECT query that finds the single most expensive expense for each user (hint: use SUM, GROUP BY user_id, and ORDER BY). Look up GROUP BY if you need to — Session 6 will formalise it.

Connect the dots — Session 5 introduces JOINS — combining the users and expenses tables into a single result set so you can see names alongside expense amounts.

Going further — Search 'SQL window functions' — functions like ROW_NUMBER() and RANK() allow ranking within a result set and are widely used in data analytics roles in India.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a SQL instructor. Explain SELECT, WHERE, ORDER BY, and aggregate functions (COUNT, SUM, AVG) in simple Hinglish. Use the example of a student tracking monthly expenses in different categories — Food, Transport, Books. Show how to find the category where the most money was spent. Keep it under 150 words.

SESSION

05

60 MIN · SQL QUERYING

Joins and Relationships

◆ Learning Objectives

- Explain why JOINS are necessary when data is split across multiple related tables
- Write an INNER JOIN query to retrieve only rows that have a match in both tables
- Use table aliases to write shorter and more readable JOIN queries
- Apply a WHERE filter on top of a JOIN to narrow results
- Distinguish between LEFT JOIN and RIGHT JOIN and when to use each
- Combine the users and expenses tables to generate a full named expense report

01 Why JOINS Exist

In a well-designed relational database, data is split across multiple tables — users in one table, expenses in another. This eliminates duplication (you store a user's name once, not on every expense row), but it means a single question like 'What did Riya spend on food?' now needs data from two tables at the same time. JOINS are the SQL mechanism that pulls related rows from multiple tables into a single result.

CONCEPT — What a JOIN Does

A JOIN temporarily combines rows from two or more tables based on a matching condition — usually a primary key in one table matching a foreign key in another. The combined result exists only for that query; nothing in the database is permanently merged. This is why relational databases can stay clean and normalised while still answering complex cross-table questions.

Without JOIN — Two Queries

Step 1: `SELECT user_id FROM users WHERE name = 'Riya Shah';` — returns `user_id = 1`. Step 2: Manually note the ID. Step 3: `SELECT * FROM expenses WHERE user_id = 1;` — two separate round-trips, manual copy-paste of IDs, error-prone at scale.

With JOIN — One Query

`SELECT u.name, e.category, e.amount FROM users u INNER JOIN expenses e ON u.user_id = e.user_id WHERE u.name = 'Riya Shah';` — one query, one result, no manual steps. Scales to millions of rows with no extra work.

02 INNER JOIN — Matching Records Only

INNER JOIN returns only the rows where the join condition is satisfied in both tables. If a user has no expenses, they will not appear in the result. If an expense somehow has no matching user (which your FOREIGN KEY constraint prevents), it will not appear either. INNER JOIN is the most commonly used join type — it answers 'show me rows that exist in both tables.'

```
SELECT users.name, expenses.category, expenses.amount
```

Specify columns from both tables using full table names

```
FROM users
```

Start from the users table

```
INNER JOIN expenses ON users.user_id = expenses.user_id;
```

Match rows where user_id exists in both tables

◆ Cleaner Version — Table Aliases

```
SELECT u.name, u.city, e.category, e.amount, e.expense_date
```

Short aliases make column references much cleaner

```
FROM users u
```

Alias 'users' as 'u' for the rest of this query

```
INNER JOIN expenses e ON u.user_id = e.user_id
```

Alias 'expenses' as 'e' — join on matching user_id

```
WHERE e.amount > 400;
```

Filter to show only expenses above Rs 400 with user names

TRY THIS LIVE

Run the aliased INNER JOIN above. You should see Riya, Arjun, and Priya's records (expenses above Rs 400). Then remove the WHERE clause and run again — now you will see all 5 expenses each paired with their user's name and city. This is what every order history page in every app shows you.

03 LEFT JOIN and RIGHT JOIN

INNER JOIN only returns matched rows. But what if you want all users — even those who have not yet logged any expenses? LEFT JOIN returns every row from the left table (the one after FROM), plus matching rows from the right table. Where there is no match, the right-table columns appear as NULL. RIGHT JOIN does the reverse — every row from the right table, with NULLs on the left when there is no match.

LEFT JOIN

All rows from the left (FROM) table are included. If no matching row exists in the right table, those columns show as NULL. Example: All 5 users appear — Sneha shows NULL in expense columns if she has no expenses logged yet. Most commonly used join after INNER JOIN.

RIGHT JOIN

All rows from the right (JOIN) table are included, with NULLs on the left where no match exists. Rarely used in practice — most queries are rewritten as a LEFT JOIN by swapping the table order. The results are logically equivalent; LEFT JOIN is simply the convention.

◆ LEFT JOIN — All Users, Even Those With No Expenses

```
SELECT u.name, u.city, e.category, e.amount
FROM users u
LEFT JOIN expenses e ON u.user_id = e.user_id;
```

Right table — expense columns show NULL if no match

Columns from both tables — expense columns will be NULL for unmatched users

Left table — all rows from users will appear

◆ RIGHT JOIN — All Expenses, Even Those With No Matching User

```
SELECT u.name, e.category, e.amount
FROM expenses e
RIGHT JOIN users u ON u.user_id = e.user_id;
```

Columns from both tables

Left table is now expenses

All users appear — equivalent to the LEFT JOIN above with tables swapped

PRO TIP — Which JOIN to Use?

Use INNER JOIN when you only care about rows that exist in both tables (most queries). Use LEFT JOIN when you want all rows from the primary table even if the related table has no match — for example, 'show all users including those who have not placed an order yet'. Avoid RIGHT JOIN; it is almost always cleaner to flip the table order and use LEFT JOIN instead.

04 Trainer Demo: Full Named Expense Report

The query below is what a real expense-tracker dashboard would run to display a user-facing transaction history: names from the users table, category and amount from the expenses table, filtered by date and sorted by amount. This is a production-grade query built entirely from what you have learned in Sessions 1 through 5.

```
SELECT u.name      AS student,
       u.city      AS location,
       e.category,
       e.amount     AS rupees,
       e.expense_date
FROM users u
INNER JOIN expenses e ON u.user_id = e.user_id
WHERE e.expense_date >= '2025-06-01'
ORDER BY e.amount DESC;
```

User's name — aliased to 'student' for a clean report heading

User's city

Expense category — Food, Transport, Books, etc.

Expense amount — aliased for readability

Date the expense was made

Primary table — users

Join expenses to users via the shared user_id

Only expenses from June 2025 onwards

Most expensive expense at the top

CASE STUDY — Zomato & Multi-Table JOINS

Context — Zomato is India's largest food delivery platform, headquartered in Gurugram, serving 500+ cities including Ahmedabad, Surat, and Vadodara. Their database holds separate tables for customers, restaurants, orders, order_items, and delivery_partners.

Challenge — When a user opens their Zomato order history, the app needs data from multiple tables simultaneously: the customer's name, the restaurant's name and cuisine, what items were ordered, and the delivery status. Fetching each table separately would require 4+ round-trips and manual stitching — far too slow for a mobile experience.

What they did — Zomato's backend runs JOIN queries across 4–5 tables for every order history page load. INNER JOINS link confirmed orders to their restaurants and items. LEFT JOINS handle in-progress orders that may not yet have a delivery record — the delivery columns appear as NULL until a rider is assigned, rather than the order disappearing from the list.

Result — Zomato serves real-time order history, live rider tracking, and personalised restaurant recommendations — all powered by multi-table JOIN queries running in milliseconds across hundreds of millions of rows.

Takeaway — Every screen in every delivery app that shows 'Order from Restaurant X — delivered by Driver Y — Rs 349' is a multi-table JOIN. You built the foundation of that today.

Source: Zomato Engineering Blog (blog.zomato.com); Zomato DRHP filing, SEBI, 2021

ZOMATO'S REACH**500+**

Indian cities where Zomato's JOIN queries link customer, restaurant, and order tables for every page load. Source: Zomato Annual Report 2022–23

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Database Management System — SQL JOINS: Inner, Left, Right (IIT Kharagpur). Navigate to the Joins section on the course page and confirm lecture numbers before sharing with students.

Link — <https://nptel.ac.in/courses/106105175>

“A JOIN does not merge tables — it asks them a question together and returns a combined answer.”

WHERE TO GO NEXT

You can now pull data across related tables — next you will group, filter, and nest those queries for deeper analysis.

Explore deeper — Read: 'MySQL JOIN Syntax' — dev.mysql.com/doc/refman/8.0/en/join.html

Try on your own — Write a LEFT JOIN that lists all 5 users alongside their total expenses (SUM). Users with no expenses should show NULL or 0 in the total column. Combine what you know from Sessions 4 and 5.

Connect the dots — Session 6 introduces GROUP BY, HAVING, and subqueries — the tools for slicing and nesting the JOIN results you built today into even more powerful reports.

Going further — Research 'FULL OUTER JOIN' and 'CROSS JOIN' — two more join types not covered in this session. MySQL does not support FULL OUTER JOIN natively, so look up how to simulate it using UNION.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a SQL instructor. Explain INNER JOIN, LEFT JOIN, and RIGHT JOIN in simple Hinglish using the example of matching students in a hostel register with their mess bill records. Tell me what happens to a student who has no mess bill — which JOIN type shows them and which hides them? Keep it under 150 words.

SESSION

06

60 MIN · SQL QUERYING

Grouping and Subqueries

◆ Learning Objectives

- Use GROUP BY to collapse rows that share a common value into a single summary row
- Apply COUNT, SUM, and AVG to grouped results to produce category-level reports
- Filter grouped results using HAVING — and explain why WHERE cannot be used instead
- Clearly state the difference: WHERE filters rows before grouping; HAVING filters groups after
- Write a non-correlated subquery inside a WHERE clause using IN
- Combine GROUP BY, HAVING, JOIN, and a subquery to answer complex analytical questions

01 GROUP BY — Collapsing Rows Into Summaries

In Session 4 you saw SUM and AVG produce a single number from all rows. GROUP BY takes this further — it splits rows into groups first, then applies the aggregate to each group separately. Instead of 'total rupees spent across all expenses', you get 'total rupees spent per category'. Every dashboard in every app — from Groww portfolio breakdowns to HDFC bank statements — runs on GROUP BY.

CONCEPT — GROUP BY

GROUP BY groups rows that share the same value in the specified column, then collapses each group into one summary row. Every column in SELECT that is not an aggregate (COUNT, SUM, AVG...) must appear in the GROUP BY clause — MySQL 8.0 enforces this strictly. Think of GROUP BY as 'sort and collapse.'

```
-- Total and count of expenses per category

SELECT category,
       COUNT(*) AS num_entries,
       SUM(amount) AS total_spent
FROM expenses
GROUP BY category
ORDER BY total_spent DESC;
```

GROUP BY collapses all rows sharing the same category into one

Category must be in GROUP BY since it is not an aggregate

Count of expense rows per category

Total rupees per category

One output row per unique category value

Highest-spending category first

```
-- Total spend per user – identifies your highest spenders

SELECT user_id,
       COUNT(*) AS num_expenses,
       SUM(amount) AS total_spent,
       AVG(amount) AS avg_per_expense
```

user_id is in GROUP BY so it is safe to SELECT

How many expenses this user has logged

Total rupees across all their expenses

Their typical expense size

```
FROM expenses
GROUP BY user_id
ORDER BY total_spent DESC;
```

One row per user

Biggest spender at the top

02 HAVING — Filtering Groups After Aggregation

You cannot use WHERE to filter on aggregate results — WHERE runs before GROUP BY, so SUM(amount) does not yet exist when WHERE is evaluated. HAVING runs after GROUP BY, giving you access to the aggregated values. Use HAVING any time your filter condition involves COUNT, SUM, AVG, MIN, or MAX.

CONCEPT — HAVING Syntax

SELECT col, AGG(col) FROM table GROUP BY col HAVING AGG(col) condition. HAVING sits after GROUP BY and before ORDER BY. You can combine WHERE and HAVING in the same query: WHERE filters raw rows first, then GROUP BY aggregates them, then HAVING filters the groups.

```
-- Categories where total spend exceeds Rs 500
```

HAVING filters out categories totalling Rs 500 or less

```
SELECT category, SUM(amount) AS total
FROM expenses
GROUP BY category
HAVING SUM(amount) > 500;
```

Only categories whose summed total passes this threshold appear

```
-- WHERE + GROUP BY + HAVING in one query
```

```
SELECT category, SUM(amount) AS total
FROM expenses
WHERE expense_date >= '2025-06-01'
GROUP BY category
HAVING SUM(amount) > 300;
```

WHERE runs first, GROUP BY next, HAVING last

Step 1 — remove rows before June 2025 from the dataset
Step 2 — group remaining rows by category
Step 3 — keep only groups whose June total exceeds Rs 300

WHERE — Filters Before Grouping

Applied before GROUP BY runs. Works on individual row values. Cannot reference aggregates (SUM, COUNT...). Removes rows from the dataset before any collapsing happens. Example: WHERE city = 'Ahmedabad' removes non-Ahmedabad user rows before they are grouped.

HAVING — Filters After Grouping

Applied after GROUP BY has collapsed rows into groups. Works on aggregate values. Cannot be replaced by WHERE when the condition involves COUNT, SUM, AVG. Removes entire groups from the result. Example: HAVING SUM(amount) > 500 removes categories whose total is under Rs 500.

03 Subqueries — Queries Inside Queries

A subquery is a SELECT statement nested inside another SQL statement. The inner query runs first and its result is used by the outer query. Subqueries make it possible to answer two-step questions in one statement — for example: 'Show me the names of users who have at least one expense above Rs 500' — without manually noting IDs between two separate queries.

CONCEPT — Subquery Locations

Subqueries can appear in three places. In WHERE: filters the outer query using the inner result — most common. In FROM: the inner result acts as a temporary table (called a derived table). In SELECT: returns a single value alongside each row (scalar subquery). This session covers WHERE and FROM subqueries.

```
-- Subquery in WHERE — find users with an expense above Rs 500
```

Inner query runs first, returns a list of user_ids

```
SELECT name, city FROM users
```

Outer query uses that list to filter users

```
WHERE user_id IN (
```

IN checks if user_id matches any value in the inner result

```
SELECT user_id FROM expenses WHERE amount > 500
```

Inner query: return user_ids who have an expense over Rs 500

```
);
```

Arjun (680) and Priya (850) match — their names and cities appear

```
-- Subquery in FROM — derived table (temporary result set)
```

Inner query produces a table-like result; outer query reads from it

```
SELECT u.name, u.city, spend.total_spent
```

Combine user details with the aggregated spend from the inner query

```
FROM users u
```

```
JOIN (
```

Start of the derived table subquery

```
SELECT user_id, SUM(amount) AS total_spent
```

Inner: aggregate total per user

```
FROM expenses GROUP BY user_id
```

```
) AS spend ON u.user_id = spend.user_id
```

Alias the derived table as 'spend' and join it like a regular table

```
ORDER BY spend.total_spent DESC;
```

Show highest spenders at the top

04 Trainer Demo: Find Users Averaging Over Rs 300 Per Expense

```
SELECT u.name,  
       u.city,
```

User name — must be in GROUP BY

City — must be in GROUP BY

<code>COUNT(e.expense_id) AS num_expenses,</code>	How many expenses this user has
<code>AVG(e.amount) AS avg_per_expense</code>	Their average expense size — used in HAVING filter below
<code>FROM users u</code>	
<code>INNER JOIN expenses e ON u.user_id = e.user_id</code>	
Join to bring expense data alongside user names	
<code>GROUP BY u.user_id, u.name, u.city</code>	One row per user — all non-aggregate columns listed here
<code>HAVING AVG(e.amount) > 300</code>	Only include users whose average expense exceeds Rs 300
<code>ORDER BY avg_per_expense DESC;</code>	Highest average first — Priya (Rs 850) and Arjun (Rs 680) in our dataset

TRY THIS LIVE

Run the demo query above. You should see Arjun (avg Rs 680) and Priya (avg Rs 850) — Riya's average is Rs 285 ($450+120 \div 2$) so she does not qualify. Then change `HAVING AVG > 300` to `HAVING AVG > 200` and run again — Riya now appears. This is how every loyalty programme identifies premium users.

CASE STUDY — NPCI & UPI Fraud Detection Using GROUP BY

Context — NPCI (National Payments Corporation of India) operates the UPI payment infrastructure, processing transactions for 400+ million users across PhonePe, Paytm, Google Pay, and BHIM across every Indian state.

Challenge — With 131+ billion transactions in FY 2023-24, NPCI must identify suspicious patterns in real-time — users who transfer to an unusually high number of new contacts in one day, or whose total daily outflow suddenly spikes far beyond their normal range.

What they did — NPCI's fraud detection layer uses `GROUP BY` on `user_id` and `transaction_date`, then applies `HAVING` to flag any user whose `COUNT` of distinct recipients in 24 hours exceeds a threshold, or whose `SUM` of outgoing transfers crosses a risk boundary. Subqueries then pull the full profiles of flagged users for secondary review by the risk team.

Result — UPI fraud rates remain among the lowest of any payment system globally — around 0.003% of transaction volume — attributed in part to real-time `GROUP BY` pattern-matching on aggregated transaction data.

Takeaway — Every `HAVING` clause you write today is the same mechanism NPCI uses to protect 400 million Indian users from payment fraud — just at a different scale.

Source: NPCI Annual Report 2023–24; RBI Report on Currency and Finance 2023

UPI TRANSACTIONS GROUPED DAILY

131 Bn+

UPI transactions in FY 2023-24 — each contributing to the GROUP BY aggregates NPCI uses for fraud detection. Source: NPCI Annual Report 2023-24

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Database Management System — GROUP BY, HAVING, and Nested Queries (IIT Kharagpur). Navigate to the aggregation and subquery lectures on the course page. Verify lecture numbers before sharing.

Link — <https://nptel.ac.in/courses/106105175>

“GROUP BY turns a list of raw events into a story. HAVING decides which chapters are worth telling.”

WHERE TO GO NEXT

You can now group, aggregate, filter groups, and nest queries — the full analytical toolkit is in your hands.

Explore deeper — Read: 'MySQL GROUP BY and HAVING' — dev.mysql.com/doc/refman/8.0/en/group-by-handling.html

Try on your own — Write a query that finds categories where average expense > Rs 400 AND total transactions >= 2. You need both HAVING conditions joined with AND.

Connect the dots — Session 7 shifts from querying to design — you will add constraints to protect data integrity and create VIEWS to save your most-used queries as named virtual tables.

Going further — Research 'correlated subquery vs non-correlated subquery' — a correlated subquery references the outer query in its WHERE clause and re-runs for every outer row, making it powerful but potentially slow on large tables.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a SQL instructor. Explain the difference between WHERE and HAVING in simple Hinglish. Use the example of a student grouping their monthly expenses by category and wanting to find categories where they spent more than Rs 500 total. Tell me why WHERE fails and HAVING works in this case. Keep it under 150 words.

SESSION

07

60 MIN · SQL DESIGN

Constraints & Views

◆ Learning Objectives

- Define and distinguish NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, DEFAULT, and CHECK constraints
- Explain why enforcing rules at the database level is safer than relying on application code alone
- Add constraints to an existing table using ALTER TABLE ADD CONSTRAINT
- Create a VIEW that encapsulates a complex SELECT query as a reusable named object
- Query a VIEW as if it were a regular table and filter it with WHERE
- Build ActiveUserView — a view showing users with two or more logged expenses

01 Constraints — Rules That Guard Your Data

A constraint is a rule enforced at the database level — it applies every time a row is inserted or updated, regardless of which application, script, or person sends the data. If application code has a bug and tries to insert an empty email or a duplicate Aadhaar number, the database rejects it before any damage is done. Constraints are the database's immune system.

Constraint	What It Prevents or Ensures	Example Syntax
NOT NULL	Saving a row without a value in this column	name VARCHAR(100) NOT NULL
UNIQUE	Two rows having the same value in this column	email VARCHAR(150) UNIQUE
PRIMARY KEY	Duplicates AND NULLs — uniquely identifies every row	user_id INT PRIMARY KEY
FOREIGN KEY	A value that has no matching row in the referenced table	FOREIGN KEY (user_id) REFERENCES users(user_id)
DEFAULT	NULL when no value is given — substitutes a default instead	created_at DATETIME DEFAULT CURRENT_TIMESTAMP
CHECK	Values that violate a logical rule	CHECK (amount > 0)

CONCEPT — Database vs Application Enforcement

Application code can be bypassed — a developer runs a script directly, an API endpoint has a bug, or a third-party integration skips validation. Database constraints cannot be bypassed by any path. Even a raw MySQL command-line INSERT must obey them. The rule: validate in your application for good UX; enforce in

your database for guaranteed correctness.

02 Adding Constraints to Existing Tables

Constraints can be added during table creation (inside CREATE TABLE) or added later using ALTER TABLE. Adding a constraint to an existing table with data will fail if any current rows violate the rule — MySQL checks all existing data before applying the constraint. This is actually useful: it surfaces hidden data quality issues you were not aware of.

```
-- Add a named UNIQUE constraint on email      Naming constraints makes them easier to drop later
ALTER TABLE users
ADD CONSTRAINT uq_user_email UNIQUE (email);
    Rejects any INSERT or UPDATE that duplicates an existing email
-- Add a CHECK constraint – amount must be positive
ALTER TABLE expenses
ADD CONSTRAINT chk_positive_amount CHECK (amount > 0);
    Rejects any expense row with 0 or negative amount
-- Drop a named constraint when requirements change
ALTER TABLE expenses DROP CONSTRAINT chk_positive_amount;
    Named constraints can be cleanly removed without rebuilding the table
```

WARNING — Constraint Fails If Existing Data Violates It

ALTER TABLE ADD CONSTRAINT checks all existing rows. If even one row in expenses has amount = 0 or NULL, adding CHECK (amount > 0) will fail with an error. Fix the bad data first with UPDATE, then add the constraint. This is the correct workflow — the constraint is revealing a data quality issue you need to resolve.

03 VIEWS — Save Your Best Queries as Named Objects

A VIEW is a saved SELECT query that you can reference by name, just like a table. Every time you query the VIEW, MySQL runs the underlying SELECT and returns a fresh result. VIEWS do not store data — they store the query definition. They are used to: hide complexity (replace a 6-line JOIN with a one-word name), restrict access (expose only certain columns to certain users), and keep dashboards consistent (everyone queries the same VIEW instead of writing different versions of the same query).

CONCEPT — Creating and Querying a VIEW

CREATE VIEW view_name AS SELECT ... creates the view. SELECT * FROM view_name; queries it exactly like a table. CREATE OR REPLACE VIEW re-defines an existing view without dropping it. DROP VIEW view_name; removes it. The underlying tables are never affected by these operations — only the stored query definition changes.

-- Create a view: user summary with expense count and total

The SELECT inside is the query that runs every time you use this view

```
CREATE VIEW user_expense_summary AS
```

Name the view — choose something descriptive and lowercase

```
SELECT u.user_id, u.name, u.city,
       COUNT(e.expense_id) AS num_expenses,
       SUM(e.amount)       AS total_spent
FROM users u
LEFT JOIN expenses e ON u.user_id = e.user_id
```

LEFT JOIN so users with no expenses still appear (with 0 count)

```
GROUP BY u.user_id, u.name, u.city;
```

One summary row per user

-- Query the view exactly like a table

The complex JOIN + GROUP BY is now invisible to the end user

```
SELECT * FROM user_expense_summary;
SELECT * FROM user_expense_summary WHERE total_spent > 500;
```

Returns all 5 users with their expense counts and totals

Add WHERE on top of the view — filters the result further

```
SELECT name, total_spent FROM user_expense_summary
```

Select only the columns you need from the view

```
ORDER BY total_spent DESC;
```

Views support ORDER BY, WHERE, and even JOINs with other tables

PRO TIP — What You Can and Cannot Do with a VIEW

You can: SELECT from a view, filter it with WHERE, JOIN it with other tables, and add ORDER BY. You cannot (in most cases): INSERT or UPDATE through a view that contains GROUP BY, HAVING, DISTINCT, or JOINs — MySQL will reject it. Simple single-table views without aggregation are sometimes updatable, but treat views as read-only in practice.

04 Trainer Demo: ActiveUserView

The lecture flow specifies building 'ActiveUserView' — a view that shows only users with more than a threshold number of transactions. This is a common pattern in product dashboards: separate active users from dormant ones to target promotions, limit plan downgrades, or trigger re-engagement messages.

```
CREATE VIEW ActiveUserView AS
```

View definition — saved permanently until dropped

```
SELECT u.user_id, u.name, u.city,
       COUNT(e.expense_id) AS transaction_count,
```

User details — all in GROUP BY below

How many expenses this user has logged

```
       SUM(e.amount)       AS total_spent
```

Their lifetime spend

```
FROM users u
INNER JOIN expenses e ON u.user_id = e.user_id
```

Only include users who have at least one expense

```
GROUP BY u.user_id, u.name, u.city
```

```
HAVING COUNT(e.expense_id) >= 2;
```

Active = 2 or more transactions — adjust threshold as needed

```
SELECT * FROM ActiveUserView;
```

In our dataset, only Riya qualifies (she has 2 expenses)

```
SELECT * FROM ActiveUserView ORDER BY total_spent DESC;
```

Order active users by who has spent the most

```
SELECT name, transaction_count
```

Expose only specific columns from the view

```
FROM ActiveUserView WHERE transaction_count >= 2;
```

Further filter the view result in the outer query

TRY THIS LIVE

Create `user_expense_summary` first, then `ActiveUserView`. Run `SELECT * FROM ActiveUserView;` — only Riya should appear (she has 2 expenses). Then go back and add two more expenses for another user (Arjun or Kabir) via `INSERT INTO expenses`, re-run the VIEW query, and watch the view result update automatically — you changed no view code, only the underlying data.

CASE STUDY — UIDAI & Database-Level Constraint Enforcement

Context — UIDAI (Unique Identification Authority of India) manages the Aadhaar biometric database — the world's largest biometric ID system with over 1.4 billion records, covering citizens across every Indian state including Gujarat.

Challenge — Each record must have a UNIQUE 12-digit Aadhaar number, a NOT NULL demographic profile, and a UNIQUE biometric hash per person. One duplicate Aadhaar number would allow identity fraud — a criminal could receive benefits, loans, or SIM cards belonging to someone else.

What they did — UIDAI enforces constraints at the database level: PRIMARY KEY on the Aadhaar number column, UNIQUE on biometric hash values, NOT NULL on all mandatory demographic fields, and FOREIGN KEY linking biometric records to demographic records. No application code path can bypass these — even a direct database INSERT is rejected if it violates a constraint.

Result — UIDAI has issued 1.4+ billion unique IDs with zero verified cases of duplicate Aadhaar number generation — a direct result of database-level constraint enforcement, not just application-layer validation.

Takeaway — Constraints enforce rules at the data layer — the deepest, most reliable layer. Application validation improves user experience; database constraints guarantee correctness regardless of how the data arrives.

Source: UIDAI Annual Report 2023–24; Ministry of Electronics and IT, Government of India

AADHAAR DATABASE SCALE

1.4 Bn+

Aadhaar records with UNIQUE and NOT NULL constraints enforced at the database level — zero verified duplicates.

Source: UIDAI Annual Report 2023-24

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Database Management System — Integrity Constraints and Views (IIT Kharagpur). Navigate to the constraints and views section on the course page. Verify lecture numbers before sharing with students.

Link — <https://nptel.ac.in/courses/106105175>

“Application code validates for the user. Database constraints enforce for everyone — including future developers who forgot to add the check.”

WHERE TO GO NEXT

Your schema is now protected by constraints and your best queries are saved as views — next, you will embed logic directly into the database.

Explore deeper — Read: 'MySQL CREATE VIEW' — dev.mysql.com/doc/refman/8.0/en/create-view.html

Try on your own — Create a view called HighSpendView that shows users with total_spent > Rs 500. Query it. Then add a new expense that pushes another user over Rs 500 and re-query — the view should update automatically.

Connect the dots — Session 8 introduces stored procedures and PL/SQL block structure — programming logic (variables, IF, loops) that lives inside the database itself rather than in external application code.

Going further — Research 'materialized views' — unlike standard VIEWS which re-run the query each time, materialised views pre-cache the result for speed. PostgreSQL supports them natively; MySQL does not, but they are widely used in enterprise databases.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a database instructor. Explain the difference between a VIEW and a TABLE in simple Hinglish. Use the example of a college admin who needs a quick report of students who have attended more than 75% of classes. Tell me why a VIEW is better than running the same JOIN query every time. Keep it under 150 words.

SESSION

08

60 MIN · PL/SQL

Introduction to PL/SQL

◆ Learning Objectives

- Explain what PL/SQL is and why procedural logic in the database is useful
- Identify the four sections of a PL/SQL block: DECLARE, BEGIN, EXCEPTION, END
- Create a MySQL stored procedure using DELIMITER, CREATE PROCEDURE, BEGIN, and END
- Declare and use local variables with DECLARE and SET inside a procedure
- Apply IF-ELSEIF-ELSE control structures to make decisions inside a procedure
- Write a WHILE loop and build a stored procedure that calculates a user's total spend

01 What Is PL/SQL and Why Use It?

SQL is declarative — you describe what you want and the database figures out how to get it. But some tasks need step-by-step logic: 'if this amount is above Rs 5,000, apply a discount; otherwise, check the category; loop through each account and add interest.' PL/SQL (Procedural Language for SQL) adds variables, conditions, and loops to SQL — turning the database from a data store into a computing environment.

CONCEPT — PL/SQL Block Structure

Every PL/SQL block has four sections. DECLARE: define variables, cursors, and exceptions used in this block. BEGIN: the main body — SQL statements and procedural logic. EXCEPTION: error handlers — what to do if something goes wrong (optional but important). END: closes the block. In MySQL, stored procedures follow the same pattern using DELIMITER, CREATE PROCEDURE, and DECLARE inside BEGIN...END.

Plain SQL — One Statement at a Time

No variables. No conditional logic. No loops. Must be rewritten for every different input. Example: `SELECT SUM(amount) FROM expenses WHERE user_id = 1` — hardcoded to user 1. To get user 2's total, you write a second query.

Stored Procedure — Reusable Logic in the Database

Accepts parameters. Uses variables, IF, and WHILE. Runs repeatable logic from a single CALL. Example: `CALL get_user_summary(1)` and `CALL get_user_summary(2)` both use the same procedure — different inputs, same tested logic, guaranteed consistent output.

02 Stored Procedure Structure in MySQL

MySQL does not use Oracle's DECLARE/BEGIN/EXCEPTION/END keyword directly at the top level, but the same logical structure exists inside CREATE PROCEDURE. The key step before writing any procedure is changing the statement delimiter from semicolon to `//` — this stops MySQL from treating each semicolon inside the procedure as the end of the entire statement.

<code>DELIMITER //</code>	Change the statement terminator from ; to // for this block
<code>CREATE PROCEDURE hello_db()</code>	Procedure name — no parameters in this example
<code>BEGIN</code>	Equivalent to PL/SQL BEGIN — start of the procedure body
<code>SELECT 'Hello from MySQL Stored Procedure!' AS message;</code>	
Any valid SQL can go here	
<code>END //</code>	Close the procedure body — // acts as the terminator
<code>DELIMITER ;</code>	Restore the semicolon as the standard statement terminator
<code>CALL hello_db();</code>	Execute the procedure — returns the SELECT result

03 Variables, Parameters, and IF Control Structures

IN parameters are values passed into the procedure when it is called. DECLARE creates a local variable that exists only inside the procedure. SET assigns a value to a declared variable. IF-ELSEIF-ELSE adds conditional branching — the procedure takes different paths based on the data. All declared variables go at the very start of the BEGIN block, before any SQL or logic.

<code>DELIMITER //</code>	
<code>CREATE PROCEDURE classify_expense(IN p_amount DECIMAL(10,2))</code>	
IN parameter — value passed in by the caller	
<code>BEGIN</code>	PL/SQL equivalent: BEGIN
<code>DECLARE v_level VARCHAR(20);</code>	DECLARE a local variable — PL/SQL DECLARE section equivalent
<code>IF p_amount < 200 THEN</code>	Conditional — runs this branch if amount is under Rs 200
<code>SET v_level = 'Low';</code>	Assign a value to the local variable
<code>ELSEIF p_amount BETWEEN 200 AND 600 THEN</code>	Second condition — between Rs 200 and Rs 600
<code>SET v_level = 'Medium';</code>	
<code>ELSE</code>	Catches everything above Rs 600
<code>SET v_level = 'High';</code>	
<code>END IF;</code>	Close the IF block
<code>SELECT p_amount AS amount, v_level AS spend_level;</code>	
Return the classification as a result set	
<code>END //</code>	
<code>DELIMITER ;</code>	
<code>CALL classify_expense(450.00);</code>	Test: passes Rs 450 -> expects 'Medium'

TRY THIS LIVE

Create the classify_expense procedure, then call it three times: CALL classify_expense(120.00); -> Low.

CALL classify_expense(450.00); -> Medium. CALL classify_expense(850.00); -> High. Verify each output matches. Then modify the thresholds — change 200 to 300 — and observe how the classification shifts.

04 WHILE Loops and Cursors

A WHILE loop repeats a block of statements as long as a condition is true. Loops are used when you need to process records one by one — for example, computing interest for each account in sequence. A cursor is a database object that lets you iterate through a SELECT result row by row inside a procedure, similar to a for-loop over a list.

```
DELIMITER //
CREATE PROCEDURE count_up(IN p_max INT)    Counts from 1 to p_max, one step at a time
BEGIN
  DECLARE v_counter INT DEFAULT 1;        Initialise counter at 1
  WHILE v_counter <= p_max DO              Loop condition — runs as long as counter is within range
    SELECT v_counter AS step;              Print the current step number
    SET v_counter = v_counter + 1;          Increment counter — always ensure the loop will terminate
  END WHILE;                               Close the loop
END //
DELIMITER ;
CALL count_up(3);                          Returns three result sets: 1, 2, 3
```

CONCEPT — Cursors (Overview)

A cursor is a pointer that moves through a SELECT result one row at a time. DECLARE CURSOR defines it; OPEN starts it; FETCH reads the current row into variables; CLOSE releases it. In MySQL, cursors must be declared before other variables in the DECLARE section and always paired with a NOT FOUND handler to detect when the last row is reached. Cursors are covered in depth in advanced PL/SQL courses — for now, understand their purpose: row-by-row processing when a single aggregate query is not sufficient.

05 Trainer Demo: User Spend Summary Procedure

The demo procedure below takes a user_id as input, fetches the user's name, counts their expenses, sums the total, and returns a clean one-row summary. This is the same pattern SBI uses for account statement generation — pass in an account number, get a formatted summary back.

```
DELIMITER //
CREATE PROCEDURE get_user_summary(IN p_user_id INT)
  Takes a user_id as input — works for any user in the table
BEGIN
```

```

DECLARE v_name VARCHAR(100);
DECLARE v_total DECIMAL(10,2) DEFAULT 0.00;

DECLARE v_count INT DEFAULT 0;
SELECT name INTO v_name
FROM users WHERE user_id = p_user_id;
SELECT SUM(amount), COUNT(*)
INTO v_total, v_count
FROM expenses WHERE user_id = p_user_id;
SELECT v_name          AS student,
       v_count          AS num_expenses,
       COALESCE(v_total, 0) AS total_spent;

END //
DELIMITER ;

```

Will hold the user's name

Will hold total spend — default 0 in case user has no expenses

Will hold number of expenses

INTO assigns the query result directly to a variable

Fetch this user's name

Fetch aggregates in one query

Assign both results to their respective variables

Return a formatted summary row

COALESCE replaces NULL with 0 if user has no expenses

```

CALL get_user_summary(1);   Riya: 2 expenses, total Rs 570 (450+120)
CALL get_user_summary(2);   Arjun: 1 expense, total Rs 680
CALL get_user_summary(5);   Sneha: 0 expenses — total_spent returns 0 due to COALESCE

```

CASE STUDY — SBI & PL/SQL for Interest Calculation

Context — State Bank of India (SBI) is India's largest public sector bank, serving over 50 crore customers across 22,000+ branches. Their Oracle database handles daily interest calculation, EMI processing, and monthly statement generation for crores of accounts.

Challenge — Computing monthly interest individually for 50 crore accounts would require 50 crore separate SQL queries — each fetching the balance, applying the rate, and updating the account. Running this as separate application-level loops would be extremely slow and prone to partial failures.

What they did — SBI uses Oracle PL/SQL stored procedures for batch account processing: DECLARE variables for rate, principal, and accrued interest; use a cursor to iterate through all eligible accounts; compute interest with a formula inside the loop; UPDATE each account; EXCEPTION handlers catch and log any account that fails without halting the entire batch.

Result — Monthly interest processing and EMI scheduling run as overnight batch stored procedures, updating crores of accounts with consistent, auditable logic — without a developer rewriting the query every month.

Takeaway — Stored procedures package repeatable business logic into the database itself — the logic runs close to the data, executes faster than application-layer loops, and remains consistent across every single call.

Source: SBI Annual Report 2023–24; Oracle India Financial Services Case Studies
(oracle.com/industries/financial-services)

SBI ACCOUNTS PROCESSED BY PROCEDURES

50 Cr+

SBI customers whose accounts are managed using PL/SQL stored procedures for interest, EMI, and statement generation.

Source: SBI Annual Report 2023-24

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Database Management System — PL/SQL: Stored Procedures, Variables, Control Structures (IIT Kharagpur). Navigate to the PL/SQL section on the course page. Verify lecture numbers before sharing with students.

Link — <https://nptel.ac.in/courses/106105175>

“A stored procedure is a tested, named solution that lives next to the data — call it a million times and it behaves identically every single time.”

WHERE TO GO NEXT

You can now write reusable logic inside your database — next you will make that logic bulletproof with transactions and triggers.

Explore deeper — Read: 'MySQL Stored Procedures' — dev.mysql.com/doc/refman/8.0/en/stored-programs-defining.html

Try on your own — Modify `get_user_summary` to also return the category where the user spent the most (use a subquery inside the procedure). Hint: `SELECT category INTO v_top_category FROM expenses WHERE user_id = p_user_id ORDER BY amount DESC LIMIT 1.`

Connect the dots — Session 9 covers COMMIT, ROLLBACK, and TRIGGERS — the mechanisms that make database operations reliable even when networks fail, servers crash, or two users write at the same time.

Going further — Research 'MySQL error handling with DECLARE HANDLER' — the equivalent of PL/SQL EXCEPTION blocks. DECLARE EXIT HANDLER FOR SQLEXCEPTION catches any SQL error inside a procedure and lets you log it or ROLLBACK safely.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a PL/SQL instructor. Explain what a stored procedure is in simple Hinglish. Use the example of a bank employee who runs the same interest calculation report every month. Tell me why storing this logic as a procedure is better than writing the SQL from scratch each time. Keep it under 150 words.

SESSION

09

60 MIN · SQL ADVANCED

Transactions & Error Handling

◆ Learning Objectives

- Define a database transaction and explain the four ACID properties in plain terms
- Wrap multiple SQL statements in a transaction using `START TRANSACTION`
- Use `COMMIT` to permanently save a transaction and `ROLLBACK` to undo it entirely
- Create a `SAVEPOINT` inside a transaction to enable partial rollback
- Write a trigger that fires `AFTER INSERT` on expenses to automatically update a user's total spend
- Test the trigger by inserting a new expense and verifying the users table updates automatically

01 Transactions — All or Nothing

A transaction is a group of SQL operations treated as a single unit. Either all of them succeed and are saved, or none of them are saved. This is the 'all or nothing' guarantee. When you send Rs 500 via UPI, two things must happen together: Rs 500 leaves your account AND Rs 500 arrives in the receiver's account. A database transaction ensures this pair succeeds or fails as one — no half-states, no disappearing rupees.

CONCEPT — ACID Properties

Atomicity: all operations in a transaction succeed together or are all rolled back — no partial results.

Consistency: the database moves from one valid state to another — all constraints and triggers are respected. Isolation: concurrent transactions do not interfere with each other — your transfer does not see another user's mid-processing transfer. Durability: once `COMMIT`ted, the change is permanent — even a server crash immediately after cannot undo it.

WARNING — MySQL AUTOCOMMIT

MySQL defaults to `autocommit = 1`, meaning every single statement is committed immediately. To use explicit transactions with `COMMIT` and `ROLLBACK`, always start with `START TRANSACTION` — this temporarily suspends `autocommit` for that session. Alternatively, run `SET autocommit = 0` to disable it for the entire session. Never assume `autocommit` is off unless you explicitly set it.

02 COMMIT and ROLLBACK

`COMMIT` permanently saves all changes made since `START TRANSACTION`. `ROLLBACK` undoes all changes back to the state before `START TRANSACTION`. The golden rule: always preview your changes after each statement inside a transaction before committing. If anything looks wrong, `ROLLBACK` — it is always safer to undo and retry than to commit bad data.

```
START TRANSACTION;
INSERT INTO expenses (user_id, category, amount, expense_date)
VALUES (1, 'Entertainment', 350.00, CURDATE());
UPDATE users SET city = 'Gandhinagar' WHERE user_id = 2;
```

Begin the transaction — autocommit suspended from here

Step 1 — add a new expense for Riya

CURDATE() inserts today's date automatically

Step 2 — update Arjun's city simultaneously

-- Verify both changes look correct, then:

COMMIT;

Always check results before committing

Both changes are permanently saved together

```
START TRANSACTION;
DELETE FROM expenses WHERE user_id = 3;
DELETE FROM users WHERE user_id = 3;
-- Something looks wrong — the wrong user was targeted:
ROLLBACK;
SELECT * FROM users;
```

Attempting to remove Priya's expenses

Then remove Priya's user record

You notice the error before committing

Both DELETES are undone — neither row is affected

Verify: Priya still exists — the ROLLBACK worked

03 SAVEPOINT — Partial Rollback

SAVEPOINT creates a named checkpoint inside a running transaction. ROLLBACK TO SAVEPOINT undoes only the statements after that checkpoint, leaving earlier statements intact. This is useful in multi-step operations where you want to retry just the last step without losing all the work done before it.

```
START TRANSACTION;
INSERT INTO expenses (user_id, category, amount, expense_date)
VALUES (2, 'Transport', 150.00, CURDATE());
SAVEPOINT sp_after_transport;
INSERT INTO expenses (user_id, category, amount, expense_date)
VALUES (2, 'Food', 0.00, CURDATE());
ROLLBACK TO SAVEPOINT sp_after_transport;
-- Fix and re-insert with correct amount:
INSERT INTO expenses (user_id, category, amount, expense_date)
VALUES (2, 'Food', 320.00, CURDATE());
COMMIT;
```

Begin transaction

First insert — Arjun's Transport expense

Checkpoint — first insert is safe here

Second insert — Arjun's Food expense

Oops — amount of 0 should not be allowed

Undo only the second insert — first insert is still pending

Correct amount this time

Both inserts (Transport 150 + Food 320) are saved

04 Triggers — Automatic Reactions to DML Events

A trigger is a stored procedure that fires automatically when a specific DML event (INSERT, UPDATE, or DELETE) occurs on a table. You define it once; from that point on, every matching DML operation activates it — no application code changes required. Triggers use NEW to reference the values being inserted or updated, and OLD to reference values before the change.

CONCEPT — BEFORE vs AFTER Triggers

BEFORE triggers fire before the DML runs — useful for validation (reject a row if amount is negative) or modifying values before they are saved. AFTER triggers fire after the DML runs — useful for cascading updates to related tables (update a running total after a new row is inserted). This session uses AFTER INSERT — the new expense row is already committed when the trigger runs.

◆ Setup — Add a Spend Tracker Column to Users

```
ALTER TABLE users
ADD COLUMN total_spent DECIMAL(10,2) DEFAULT 0.00;
```

Add a column to track running total spend per user
Starts at 0 — the trigger will keep it updated automatically

◆ Create the Trigger

```
DELIMITER //
CREATE TRIGGER trg_update_total_spent
AFTER INSERT ON expenses
FOR EACH ROW
BEGIN
    UPDATE users
    SET total_spent = total_spent + NEW.amount
    WHERE user_id = NEW.user_id;
END //
DELIMITER ;
```

Name the trigger descriptively
Fires after every new row in the expenses table
Runs once per inserted row — not once per statement
Update the users table automatically
NEW.amount = the amount value from the just-inserted expense row
NEW.user_id = the user_id from the just-inserted row

◆ Test the Trigger

```
-- Check Riya's current total before the trigger fires
SELECT user_id, name, total_spent FROM users WHERE user_id = 1;
-- Should show total_spent = 0 (column was just added)

-- Insert a new expense for Riya
INSERT INTO expenses (user_id, category, amount, expense_date)
```

```
VALUES (1, 'Books', 250.00, CURDATE());
```

Trigger fires automatically after this INSERT

```
-- Verify the trigger updated users automatically
```

```
SELECT user_id, name, total_spent FROM users WHERE user_id = 1;
```

total_spent should now show 250.00 — no UPDATE was needed

TRY THIS LIVE

After creating the trigger, insert two more expenses for different users and check the users table after each one. Notice that total_spent updates automatically every time — you wrote no UPDATE, no application code, no procedure call. Now try: what happens if you DELETE an expense? The trigger does not fire on DELETE — total_spent does not decrease. That is your next challenge: write a AFTER DELETE trigger to subtract the amount.

CASE STUDY — PhonePe & ACID-Compliant UPI Transfers

Context — PhonePe is India's largest UPI payment app by transaction volume, processing over 200 crore transactions per month across 500+ million registered users, including millions in Gujarat cities like Ahmedabad, Surat, and Rajkot.

Challenge — When a user sends Rs 500, two operations must happen together: debit Rs 500 from the sender's account and credit Rs 500 to the receiver's account. If the credit step fails midway — due to a network timeout, a bank API error, or a server crash — the money must not disappear from the sender's account without reaching the receiver.

What they did — Every UPI transfer at PhonePe is wrapped in a database transaction: START TRANSACTION -> debit sender -> credit receiver -> COMMIT. If any step returns an error, ROLLBACK immediately restores both accounts to their pre-transfer state. SAVEPOINTS are used for multi-leg transactions like bill splits or group payments, where partial rollback to a checkpoint is needed without cancelling the entire transfer.

Result — PhonePe maintains near-perfect financial reconciliation — every rupee debited from one account is verifiably credited to another — even during Diwali or IPL season when crores of transactions are processed simultaneously at peak load.

Takeaway — COMMIT and ROLLBACK are not academic exercises — they are what keeps Rs 500 from vanishing every time a payment app crashes mid-transfer. You now understand the mechanism behind every UPI payment you have ever made.

Source: PhonePe Annual Report 2023–24; NPCI UPI Transaction Report 2024

ATOMICITY IN ACTION

Rs 0

Amount that 'disappears' in a properly ACID-managed UPI transfer — atomicity guarantees debits and credits always match. Source: NPCI Annual Report 2023-24

WATCH — VIDEO REFERENCE (ENGLISH)**Channel** — NPTEL (official — IITs / IISc)**Watch** — Database Management System — Transactions: ACID, COMMIT, ROLLBACK, and Triggers (IIT Kharagpur). Navigate to the transactions section on the course page. Verify lecture numbers before sharing with students.**Link** — <https://nptel.ac.in/courses/106105175>

“A transaction is a promise: either everything happens, or nothing happened. The database keeps that promise even when the network does not.”

WHERE TO GO NEXT

You now have the complete SQL toolkit — DDL, DML, SELECT, JOINS, GROUP BY, constraints, views, procedures, and transactions. Session 10 is your chance to put all of it together.

Explore deeper — Read: 'MySQL Triggers' — dev.mysql.com/doc/refman/8.0/en/trigger-syntax.html**Try on your own** — Write a AFTER DELETE trigger on expenses that subtracts the deleted row's amount from total_spent in users. Test it by deleting a row and verifying the total decreases. This completes a fully self-maintaining running total.**Connect the dots** — Session 10 is Mini Project 3 — Expense Tracker DB. You will redesign the full schema (categories, users, transactions), write CRUD scripts, build the monthly summary procedure, and bring every Module 5 concept together in one working project.**Going further** — Research 'two-phase commit (2PC)' — the protocol used when a transaction spans two separate database servers (e.g., a UPI transfer that touches your bank's DB and the receiver's bank DB simultaneously).**AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE**

Act as a database instructor. Explain COMMIT and ROLLBACK in simple Hinglish using the example of a UPI payment between two college friends — Raj sends Rs 500 to Priya for a group dinner. Tell me what happens at the database level if the transaction fails halfway. Why can't just one of the two accounts update? Keep it under 150 words.

SESSION

10

90 MIN · MINI PROJECT

Mini Project 3 – Expense Tracker DB

◆ Learning Objectives

- Design a 3-table relational schema (categories, users, transactions) from scratch
- Write CREATE TABLE statements with appropriate data types, constraints, and foreign keys
- Populate the schema with realistic sample data using multi-row INSERT statements
- Build SELECT queries that JOIN all three tables to produce named, categorised reports
- Write a stored procedure that returns a user's monthly expense summary by category
- Apply at least one VIEW and one trigger from Sessions 7 and 9 to the new schema

01 Project Brief

You are building the complete database backend for a Personal Expense Tracker — the same kind of backend that powers apps like ET Money, Walnut, and CRED in India. The project extends the two-table design from Sessions 2–9 into a production-closer three-table schema, adds a category master table, and ties everything together with CRUD scripts and a monthly summary stored procedure.

PRO TIP — Build in This Order

Always follow this sequence: (1) Create tables in dependency order — categories first, then users, then transactions (transactions has foreign keys to both). (2) Insert sample data and verify row counts. (3) Write and test SELECT queries before writing the stored procedure. (4) Add the VIEW and trigger last — they depend on working underlying tables and data.

02 Schema Design — Three Tables

The key upgrade from Sessions 2–9 is the categories table. Instead of storing category as a free-text VARCHAR in every transaction row (where 'Food', 'food', and 'FOOD' would all be treated as different categories), categories is now a master reference table. Every transaction stores a category_id — a foreign key to categories. This eliminates typos, enables consistent reporting, and makes adding new categories trivial.

CONCEPT — Why a Separate Categories Table?

Storing categories as a reference table (normalisation) means: (1) Category names are consistent — you change 'Entertainment' to 'Fun' in one row of categories and all 500 transactions using it instantly reflect the new name. (2) You can add metadata to categories — icon, budget limit, display colour — without touching the transactions table. (3) Reports GROUP BY category_id are faster on integers than on VARCHAR strings.

◆ Table 1 — Categories (No Foreign Keys)

```
CREATE TABLE categories (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(50) NOT NULL UNIQUE,
  description  VARCHAR(255)
);
```

Master reference table — create this first

Unique ID for each category

Category name — must be unique and not empty

Optional description of the category

◆ Table 2 — Users (No Foreign Keys)

```
CREATE TABLE users (
  user_id      INT AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(100) NOT NULL,
  email        VARCHAR(150) UNIQUE NOT NULL,
  city         VARCHAR(50),
  total_spent  DECIMAL(10,2) DEFAULT 0.00,
  created_at   DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Same as Sessions 2-9, with total_spent added for the trigger

Running total — maintained automatically by a trigger (Stretch Goal)

◆ Table 3 — Transactions (Foreign Keys to Both Tables)

```
CREATE TABLE transactions (
  transaction_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id        INT NOT NULL,
  category_id    INT NOT NULL,
  amount         DECIMAL(10,2) NOT NULL,
  transaction_date DATE NOT NULL DEFAULT (CURDATE()),
  notes          VARCHAR(255),
  FOREIGN KEY (user_id) REFERENCES users(user_id),
  FOREIGN KEY (category_id) REFERENCES categories(category_id),
  CONSTRAINT chk_amount CHECK (amount > 0)
);
```

Core fact table — create last, after both reference tables exist

Links to users table

Links to categories table — integer FK, not free text

Use DECIMAL — never FLOAT for money

Defaults to today if not supplied

Optional

Cannot insert a transaction for a non-existent user

Cannot insert an unrecognised category

Database-level: rejects zero or negative amounts

03 CRUD Scripts

◆ INSERT Sample Data

```
INSERT INTO categories (name, description) VALUES
```

Populate the master table first — transactions depend on it

```
('Food',          'Meals, canteen, restaurants'),
('Transport',      'Bus, auto rickshaw, ride-share'),
('Books',          'Textbooks, notes, stationery'),
('Entertainment',  'Movies, events, streaming'),
('Other',          'Miscellaneous expenses');
```

category_ids will be 1–5 due to AUTO_INCREMENT

```
INSERT INTO users (name, email, city) VALUES
```

Same five users from Sessions 3–9

```
('Riya Shah',     'riya@example.com', 'Ahmedabad'),
('Arjun Mehta',    'arjun@example.com', 'Surat'),
('Priya Patel',    'priya@example.com', 'Rajkot'),
('Kabir Joshi',    'kabir@example.com', 'Ahmedabad'),
('Sneha Desai',    'sneha@example.com', 'Vadodara');
```

```
INSERT INTO transactions (user_id, category_id, amount, transaction_date, notes)
VALUES
```

category_id references categories — use the integer IDs

```
(1, 1, 450.00, '2025-06-01', 'Canteen lunch'),
    Riya, Food (1)
(1, 2, 120.00, '2025-06-01', 'Auto rickshaw'),
    Riya, Transport (2)
(2, 1, 680.00, '2025-06-02', 'Dinner with friends'),
    Arjun, Food (1)
(3, 3, 850.00, '2025-06-03', 'Reference book purchase'),
    Priya, Books (3)
(4, 2, 200.00, '2025-06-01', 'Monthly bus pass'),
    Kabir, Transport (2)
(1, 4, 350.00, '2025-06-05', 'Movie with friends');
    Riya, Entertainment (4)
```

◆ 3-Table JOIN Report

```
SELECT u.name      AS student,           User name from users table
       u.city      AS location,
```



```

        c.name          AS category,          Category name from categories table — not the raw
                                                integer ID
        t.amount         AS rupees,          Amount from transactions table
        t.transaction_date
FROM transactions t                          Start from transactions — the central fact table
JOIN users      u ON t.user_id      = u.user_id
    Join to get user details
JOIN categories c ON t.category_id  = c.category_id
    Join to get category name instead of ID
ORDER BY t.transaction_date, u.name;        Chronological, alphabetical by name

```

◆ UPDATE and DELETE

```

UPDATE transactions SET amount = 500.00 WHERE transaction_id = 1;
    Correct Riya's canteen lunch amount
DELETE FROM transactions WHERE transaction_id = 6;
    Remove Riya's entertainment entry
SELECT * FROM transactions;                Verify — should now have 4 transactions remaining

```

TRY THIS LIVE

Build all three tables and run the 3-table JOIN report. You should see user names, cities, category names (not IDs), amounts, and dates in one clean result set. Then try adding a transaction with category_id = 99 — MySQL should reject it with a foreign key constraint error. This is your constraint enforcement working exactly as designed.

04 Monthly Summary Procedure

The stored procedure below accepts a user_id, month, and year, then returns a category-wise breakdown of that user's spending for that month — total spent per category, number of transactions, and average transaction size. This is the backend function that powers the 'Monthly Report' screen in any expense tracker app.

```

DELIMITER //
CREATE PROCEDURE monthly_summary(
    IN p_user_id INT,
    IN p_month    INT,
    IN p_year     INT
)
BEGIN
    Three IN parameters: which user, which month, which
    year
    Month as a number: 6 for June, 12 for December

```

```

DECLARE v_name VARCHAR(100);
SELECT name INTO v_name FROM users WHERE user_id = p_user_id;
SELECT v_name          AS student,
       c.name          AS category,
COUNT(*)             AS transactions,
       Number of transactions in this category for the month
SUM(t.amount)         AS total_spent,
       Total rupees in this category for the month
AVG(t.amount)         AS avg_per_transaction
       Average transaction size
FROM transactions t
JOIN categories c ON t.category_id = c.category_id
WHERE t.user_id = p_user_id
      AND MONTH(t.transaction_date) = p_month
      AND YEAR(t.transaction_date) = p_year
GROUP BY c.category_id, c.name
ORDER BY total_spent DESC;
END //
DELIMITER ;
CALL monthly_summary(1, 6, 2025);

```

Fetch and store the user's name for the report header

Include name in every row of the output

Category name via JOIN

Filter to the requested user

Filter to the requested month

Filter to the requested year

One row per category in the result

Highest-spend category first

Riya's June 2025 report — Food, Transport, Entertainment breakdown

05 Stretch Goals

STRETCH GOALS — Try These If You Finish Early

1. Create a VIEW called ActiveUserView on the new schema — users with 2+ transactions, their total and count. 2. Write an AFTER INSERT trigger on transactions that increments total_spent in users by NEW.amount. Test it by inserting a transaction and checking users immediately after. 3. Add a BEFORE INSERT trigger that rejects any transaction with amount > 50000 — an expense cap. 4. Write a query using GROUP BY and HAVING to find users whose average transaction in June 2025 exceeds Rs 300. 5. Add a budget DECIMAL column to categories and write a query showing which categories are over budget (total_spent > budget).

MODULE 5 — COMPLETE

9

SQL skill groups mastered: DDL, DML, SELECT/WHERE, ORDER BY/Aggregates, JOINS, GROUP BY/Subqueries, Constraints/Views, PL/SQL, Transactions — all applied in this project.

“You did not just learn SQL. You built the backend of a real product — the same one fintech apps in India run on.”

WHERE TO GO NEXT

Module 5 is complete. You now have a full, working relational database backend — a skill that appears in every backend, data, and full-stack developer role in India.

Explore deeper — Explore: 'SQL Exercises' on SQLZoo (sqlzoo.net) and HackerRank SQL track — both are used in Indian company hiring assessments. Practice GROUP BY, JOINS, and subqueries under timed conditions.

Try on your own — Export your entire Expense Tracker DB as a .sql file: Workbench -> Server -> Data Export -> select your database -> Export to Self-Contained File. Keep this file — it is your Module 5 project deliverable.

Connect the dots — In upcoming modules, you will connect this database to a web application backend using PHP or Node.js — the SQL you know now is what the server will run every time a user logs in, adds an expense, or views a report.

Going further — Research 'database indexing' — adding an INDEX on user_id in the transactions table makes queries dramatically faster as your data grows to millions of rows. This is the first performance optimisation every backend developer learns after writing their first schema.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a senior database developer reviewing a student's first schema. The student has built categories, users, and transactions tables for an expense tracker. Ask me 5 questions in Hinglish that test whether I have properly normalised the schema, used appropriate data types, and added the right constraints. After I answer, give me feedback on each one.

REVIEW



INTERVIEW PREP · MODULE 5

Interview Questions

◆ SE – Database Management: SQL & PL/SQL

Rehearse each answer out loud, aiming for about 60 seconds. For SQL questions, state the query and explain what each clause does — interviewers look for understanding, not just memorized syntax.

1

What is a database, and how is it different from a DBMS?

One is a collection of data, the other is software to manage it.

MODEL ANSWER

A database is an organized collection of structured data stored on disk — for example, all the customer records for an e-commerce site. A DBMS (Database Management System) is the software that manages that data, providing tools to create, read, update, and delete records. MySQL, PostgreSQL, and Oracle are all examples of DBMS software. You interact with the DBMS using SQL, and the DBMS handles the underlying file storage, indexing, and security.

2

Explain the difference between a DBMS and an RDBMS with an example.

Think about how data relationships are handled.

MODEL ANSWER

A DBMS stores data but does not enforce relationships between tables — it may use flat files or hierarchical structures. An RDBMS (Relational Database Management System) organizes data in tables with defined relationships between them using primary and foreign keys. For example, in MySQL, a `users` table can be linked to a `transactions` table through a `user\_id` foreign key, ensuring data integrity. MySQL, PostgreSQL, and Oracle are RDBMS; they enforce referential integrity and support JOIN queries across related tables.

3

What are tables, rows, and columns in a relational database?

MODEL ANSWER

A table is the primary structure for storing data — it is similar to a spreadsheet with a fixed set of columns. Each column (also called a field or attribute) defines a specific type of data that will be stored, such as `name VARCHAR(100)` or `age INT`. Each row (also called a record or tuple) is one complete entry in the table — for example, one user's full set of information. A relational database can have many related tables, all queried together using SQL.

4

What is the purpose of the CREATE command in SQL?

It's used at the start when defining structures.

MODEL ANSWER

The `CREATE` command is a DDL (Data Definition Language) statement used to create new database structures. `CREATE DATABASE mydb;` creates a new database, and `CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100));` defines a new table with its columns and data types. You only run `CREATE TABLE` once at setup — it establishes the schema. If you run it again for the same table name without `IF NOT EXISTS`, it throws an error.

5

Write an SQL statement to create a table called 'users' with columns: id (integer, primary key), name (varchar), and email (varchar).

MODEL ANSWER

The statement is: `CREATE TABLE users ( id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(150) UNIQUE );`. `INT PRIMARY KEY` ensures each row has a unique identifier and cannot be NULL. `VARCHAR(100)` sets a variable-length string up to 100 characters. Adding `NOT NULL` on name ensures every user must have a name, and `UNIQUE` on email prevents duplicate email addresses. These constraints enforce data integrity at the database level.

6

How does the ALTER command differ from the DROP command in SQL?

One changes, the other removes.

MODEL ANSWER

`ALTER` modifies the structure of an existing table without deleting its data — for example, `ALTER TABLE users ADD COLUMN phone VARCHAR(15);` adds a new column. `DROP` permanently removes a table and all its data: `DROP TABLE users;`. `DROP` is irreversible unless you have a backup, so it should always be used with caution in production. `ALTER` is used regularly during development when the schema needs to evolve.

7

Write an SQL statement to insert a new user with name 'Amit' and email 'amit@example.com' into the users table.

MODEL ANSWER

The statement is: `INSERT INTO users (name, email) VALUES ('Amit', 'amit@example.com');`. The columns are listed in parentheses after the table name, and the corresponding values follow the `VALUES` keyword in the same order. If `id` is set to `AUTO_INCREMENT`, you do not need to include it — the database assigns it automatically. Always enclose string and date values in single quotes.

8

Explain the use of the WHERE clause in SQL statements.

It helps filter which records are affected.

MODEL ANSWER

The `WHERE` clause filters which rows are affected by a `SELECT`, `UPDATE`, or `DELETE` statement. Without it, the statement operates on every row in the table — which can be dangerous for `UPDATE` and `DELETE`. For example, `DELETE FROM users WHERE name = 'Test';` removes only rows where the name is 'Test', while `DELETE FROM users;` would erase the entire table. The `WHERE` clause can use comparison operators, logical operators (`AND`, `OR`), and the `LIKE` operator for pattern matching.

9

Write an SQL query to select the names of users from the 'users' table, sorted alphabetically.

MODEL ANSWER

The query is: `SELECT name FROM users ORDER BY name ASC;`. `SELECT name` retrieves only the name column instead of all columns. `ORDER BY name` sorts the results by the name field, and `ASC` specifies ascending (A to Z) order — `DESC` would give descending order. Adding `LIMIT 10` at the end would return only the first 10 alphabetical results, which is useful for pagination.

10

What is the purpose of aggregate functions like SUM, AVG, and COUNT in SQL?**MODEL ANSWER**

Aggregate functions perform calculations on a set of rows and return a single summary value. `COUNT(*)` counts the total number of rows, `SUM(amount)` adds up all values in a column, `AVG(amount)` calculates the mean, `MAX()` finds the highest value, and `MIN()` finds the lowest. They are most powerful when combined with `GROUP BY`, which groups rows by a column before applying the function — for example, getting the total spend per user.

11

Explain the difference between INNER JOIN and LEFT JOIN with an example scenario.*Think about what happens to unmatched rows.***MODEL ANSWER**

An INNER JOIN returns only the rows where there is a matching record in both tables. A LEFT JOIN returns all rows from the left table plus any matching rows from the right table — if there is no match, the right-side columns are NULL. For example, joining `users` and `transactions`: an INNER JOIN shows only users who have at least one transaction, while a LEFT JOIN also shows users who have never made a transaction (their transaction columns would be NULL). Use LEFT JOIN when you need to include records that may not have related data.

12

Write an SQL query to join the 'users' and 'transactions' tables on user ID, showing each user's name and transaction amount.**MODEL ANSWER**

The query is: `SELECT users.name, transactions.amount FROM users INNER JOIN transactions ON users.id = transactions.user_id;`. The `ON` clause specifies the matching condition between the two tables. Using table-prefixed column names (`users.name`) avoids ambiguity when both tables have similarly named columns. You can add `WHERE`, `ORDER BY`, or `GROUP BY` clauses after the JOIN to further filter or sort the results.

13

How does the GROUP BY clause work in SQL, and when would you use it?*It's used to aggregate data by categories.***MODEL ANSWER**

GROUP BY groups rows that have the same value in one or more columns, allowing aggregate functions to be applied per group rather than across all rows. For example, `SELECT user_id, SUM(amount) FROM transactions GROUP BY user_id;` calculates the total spending separately for each user. Without GROUP BY, `SUM(amount)` would return a single total across all users. GROUP BY is essential for reports and analytics — summarizing sales by region, expenses by category, or orders by date.

14

Explain the difference between WHERE and HAVING clauses in SQL.

One filters before grouping, the other after.

MODEL ANSWER

WHERE filters individual rows before any grouping or aggregation takes place — it works on raw column values. HAVING filters groups after GROUP BY and aggregation have been applied — it works on the result of aggregate functions. For example, to find users whose total spend exceeds Rs 500: ``SELECT user_id, SUM(amount) FROM transactions GROUP BY user_id HAVING SUM(amount) > 500;``. You cannot use aggregate functions like ``SUM()`` inside a WHERE clause — that is what HAVING is for.

15

What is a constraint in SQL? Name two types of constraints you can apply to a column.

Think about rules that enforce data integrity.

MODEL ANSWER

A constraint is a rule applied to a column that restricts what data can be stored, ensuring the database stays accurate and consistent. PRIMARY KEY ensures each row has a unique, non-null identifier. FOREIGN KEY links a column to the primary key of another table, enforcing referential integrity — you cannot insert a transaction for a user_id that doesn't exist in the users table. Other constraints include NOT NULL (prevents empty values), UNIQUE (prevents duplicate values), and CHECK (enforces a custom condition like ``age > 0``).

16

How would you create a VIEW in SQL to show only users who have made more than 5 transactions?

MODEL ANSWER

The statement is: ``CREATE VIEW ActiveUserView AS SELECT users.id, users.name, COUNT(transactions.id) AS tx_count FROM users JOIN transactions ON users.id = transactions.user_id GROUP BY users.id HAVING COUNT(transactions.id) > 5;``. A VIEW is a saved query — once created, you can query it like a table: ``SELECT * FROM ActiveUserView;``. Views simplify complex queries, provide a layer of abstraction, and can restrict which columns are visible to different users.

17

Explain the basic structure of a PL/SQL block.

There are four main sections.

MODEL ANSWER

A PL/SQL block has four sections: DECLARE (optional) — where you define variables, constants, and cursors; BEGIN — where the main executable logic goes; EXCEPTION (optional) — where you handle errors gracefully; and END — which closes the block with a semicolon. For example: ``DECLARE v_total NUMBER; BEGIN SELECT SUM(amount) INTO v_total FROM transactions WHERE user_id = 1; DBMS_OUTPUT.PUT_LINE(v_total); EXCEPTION WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No data'); END;``. This structure makes database programming more powerful than plain SQL alone.

18

Explain the role of COMMIT and ROLLBACK in database transactions.*Think about saving versus undoing changes.***MODEL ANSWER**

A transaction is a sequence of SQL statements that should either all succeed or all fail together. COMMIT permanently saves all the changes made since the last commit — once committed, the changes cannot be undone. ROLLBACK undoes all changes made since the last commit, restoring the database to its previous state. This is critical for data integrity — for example, when transferring money between accounts, both the debit and credit must succeed, or both should be rolled back to avoid one account losing money without the other gaining it.

19

Describe a scenario where a trigger would be useful in a database application.*Think about automating actions after a data change.***MODEL ANSWER**

A trigger is a stored block of PL/SQL that automatically fires before or after a specific DML event (INSERT, UPDATE, or DELETE) on a table. For example, in an expense tracker, when a new expense is inserted into the `transactions` table, an AFTER INSERT trigger can automatically update the `balance` column in the `users` table — so the balance always reflects the latest total without the application code needing to run a separate UPDATE statement. Triggers ensure consistency and enforce business rules at the database level.

20

In your Expense Tracker DB mini project, how would you write a stored procedure to generate a monthly summary for each user?**MODEL ANSWER**

The procedure would accept a month and year as parameters, then use a cursor or a query to loop through each user, calculating their total expenses for that period using `SUM(amount)` with a date filter. For each user, it would output (or insert into a summary table) the user name, total spend, and category breakdown. A stored procedure is defined once and called with `EXEC monthly_summary(1, 2025);` — it runs on the database server, making it faster than sending multiple queries from the application. Always test with sample data and use EXCEPTION handling for edge cases like users with no transactions that month.